게임 프로그래밍

100을 넘어라 → 아이템 카드 배틀

게임 업그레이드 보고서

학번 / 이름 : 2022564036 / 이형원

원본소스에 대한 분석

게임 개요 및 동작 원리

게임명 : 100을 넘어라 게임 (기본 버전)

개발 언어 : C언어 (표준 라이브러리 및 Windows API 활용)

핵심 규칙 : 컴퓨터와 사용자가 번갈아 가며 1~10 사이의 숫자를 입력.
한 번 사용한 숫자는 중복해서 사용할 수 없음. (배열로 중복 체크)
입력한 숫자들의 누적 합계가 최초로 100을 넘기는 쪽이 승리.

기존 코드의 한계 및 업그레이드 방향 (도입부)

기존 코드의 아쉬운 점 : 단순한 UI 및 제한적인 게임 플레이 방식.
컴퓨터(AI)가 무작위로만 숫자를 내기 때문에 난이도가 단조로움.
입력 예외 처리(scanf 문자 입력 시 무한 루프 버그 등)의 부재.

업그레이드 (아이템 카드 배틀 버전) 예고 : 방향키 UI 도입, 특수 아이템(더블, 스킵 등) 추가, AI 난이도 도입
및 화려한 컬러 콘솔 UI 구성.

업그레이드 개요

버그 수정 (6)

업1~6 : 입력 범위 검사 추가, 문자 입력 무한 루프 해결, 화면 잔상 제거, 변수 win 초기화, 결과 위치 고정 + 창 바로 닫힘 해결, Visual Studio 컴파일 오류 해결

코드 품질 (2)

업7~8 : goto 제거, 매직 넘버 상수화

화면/연출 (14)

업9~22 : 콘솔 창 설정, 커서 숨기기, 색상 시스템, 1초 로고(스플래시) 화면, 방향키 메인 메뉴, 게임 방법 화면 개편, 카드 모양 UI, 방향키로 카드 선택, 합계 게이지 바, 화면 깜빡임 줄이기, 합계 카운트업 애니메이션, 컴퓨터 생각 중 연출, 게임 기록(로그) 창, 효과음 + 소리 ON/OFF

게임 시스템 (4)

업23~26 : 난이도 3단계 AI, 동전 던지기로 선공 정하기, 제한 시간 20초, 숫자 카드 리필 규칙

아이템 배틀 (7)

업27~33 : 아이템 카드 시스템 도입, 아이템 1: 더블 x2, 아이템 2: 합계 −10, 아이템 3: 턴 스킵, 아이템 4: 봉인, 보너스 아이템 지급, AI 아이템 전략

저장/마무리 (5)

업34~38 : 게임 저장, 이어하기(불러오기), 전적 기록 저장/보기, 엔딩 화면, 일시정지 + 다시 하기

업그레이드 1 : 입력 범위 검사 추가

[버그 수정]

이전 (문제점)

999를 입력하면 바로 이기고, -50을 넣으면 합계가 줄어들었음

변경 내용

1~10 밖의 숫자를 입력하면 경고하고 다시 입력받음
(최종판은 방향키 선택으로 아예 불가능)

효과

게임 규칙이 실제로 지켜짐

관련 코드 : user_number() / user_turn()

```
int user_turn(Game *g)
{
    Player *me = &g->p[USER];
    int row = 0, col = max_playable(me) - 1;
    int item_used = 0, key, remain, last_sec = -1;
    clock_t start = clock(), pause_start;

    draw_hand(g, row, col, item_used);
    while (1)
    {
```

업그레이드 2 : 문자 입력 무한 루프 해결

[버그 수정]

이전 (문제점)

abc를 입력하면 scanf가 실패하고 입력이 남아 무한 반복

변경 내용

scanf 반환값을 확인하고 입력 버퍼를 비움
(clear_input_buffer)

효과

프로그램이 멈추지 않음

관련 코드 : clear_input_buffer()

```
int user_turn(Game *g)
{
    Player *me = &g->p[USER];
    int row = 0, col = max_playable(me) - 1;
    int item_used = 0, key, remain, last_sec = -1;
    clock_t start = clock(), pause_start;

    draw_hand(g, row, col, item_used);
    while (1)
    {
```

업그레이드 3 : 화면 잔상 제거

[버그 수정]

이전 (문제점)

이전 입력과 오류 메시지가 화면에 그대로 남아 30처럼 보였음

변경 내용

출력 전에 그 줄을 공백으로 지우는 clear_line() 추가

효과

항상 깨끗한 화면

관련 코드 : clear_line()

```
void clear_line(int y)
{
    gotoxy(1, y);
    printf("%79s", "");
}

void print_at(int x, int y, int color, const char
*fmt, ...)
{
    va_list ap;
    gotoxy(x, y);
    set_color(color);
    va_start(ap, fmt);
    vprintf(fmt, ap);
    va_end(ap);
    set_color(C_GRAY);
}
```

업그레이드 4 : 변수 win 초기화

[버그 수정]

이전 (문제점)

win이 초기화되지 않아 정의되지 않은 동작 가능

변경 내용

int win = 0;으로 초기화, 무승부 경우도 처리

효과

예상치 못한 결과 방지

관련 코드 : game_control()

```
void game_control(void)
{
	int max=100;
	int com_array[10]={0},
user_array[10]={0}, number;
	int i, sum=0, win;
	for(i=0;i<10;i+=1)
	{
		system("cls");
		number=computer_number(i,
com_array);
		sum+=number;
		number_display(i, com_array,
user_array, sum);
		if (max<sum)
		{
			win=1;
			break;
		}
```

업그레이드 5 : 결과 위치 고정 + 창 바로 닫힘 해결

[버그 수정]

이전 (문제점)

결과가 아무 데나 찍히고 프로그램이 바로 끝남

변경 내용

gotoxy로 위치를 고정하고, 끝난 뒤 키 입력 대기 (최종판은 엔딩 화면과 메뉴 복귀)

효과

결과를 확실히 확인 가능

관련 코드 : conclusion() → ending_screen()

```
        system("cls"); // [수정] 마지막 결과 출력 시 화면을 깨끗하게 정리
        gotoxy(1, 2);
        if (check == 1)
                printf("컴퓨터가 이겼습니다! ₩n");
        else
                printf("사용자가 이겼습니다! ₩n");
}
```

업그레이드 6 : Visual Studio 컴파일 오류 해결

[버그 수정]

이전 (문제점)

VS에서 scanf C4996 오류, getch 사용 권장 안 함

변경 내용

_CRT_SECURE_NO_WARNINGS 정의, getch → _getch

효과

최신 VS에서 바로 빌드됨

관련 코드 : 파일 맨 위, get_key()

```
void intro_game(void)
{
	system("cls");
	printf("100을 넘어라 게임₩n₩n");
	printf("컴퓨터와 사용자가 번갈아가며₩n");
	printf("숫자를 입력하되 1~10사이의 ₩n");
	printf("중복되지 않는 숫자를 입력하여
₩n₩n");
	printf("최초로 100을 넘기면 승자가 됩니다.
₩n₩n");
	printf("컴퓨터가 먼저 시작합니다. ₩n");
	printf("아무키나 누르면 시작합니다.");
	getch();
}
```

업그레이드 7 : goto 제거

[코드 품질]

이전 (문제점)

goto again으로 되돌아가는 구조라 흐름 파악이 어려움

변경 내용

while / do-while 반복문과 함수로 바꿈

효과

읽기 쉽고 고치기 쉬운 코드

관련 코드 : computer_number(), user_number()

```
int computer_number(int i, int com_array[])
{
        int j, number;
        again:;
        number=rand()%10+1;
        for(j=0;j<i;j+=1)
                if (com_array[j]==number)
                        goto again;
        com_array[i]=number;
        return com_array[i];
}

int user_number(int i, int user_array[])
{
        int j, number;
        again:;
        gotoxy(1, 7);
        printf("사용자 숫자 입력후 Enter>");
        scanf("%d", &number);
        for(j=0;j<i;j+=1)
                if (user_array[j]==number)
                {
                        gotoxy(1, 8);
                        printf("중복된 숫자를 입력했습니다. ₩n");
                        goto again;
                }
        user_array[i]=number;
        return user_array[i];
}
```

업그레이드 8 : 매직 넘버 상수화

[코드 품질]

이전 (문제점)

100, 10 같은 숫자가 코드 곳곳에 직접 쓰여 있음

변경 내용

MAX_SCORE, MAX_NUM 등 #define 상수로 정리

효과

규칙을 한 줄만 고쳐 바꿀 수 있음

관련 코드 : 파일 위쪽 #define

```
void press_any_key(void)
{
	gotoxy(1, 10);
	printf("아무키나 누르시오..");
	getch();
}

void game_control(void)
{
	int max=100;
	int com_array[10]={0}, user_array[10]={0}, number;
	int i, sum=0, win;
	for(i=0;i<10;i+=1)
	{
		system("cls");
		number=computer_number(i, com_array);
		sum+=number;
		number_display(i, com_array, user_array, sum);
		if (max<sum)
		{
			win=1;
			break;
		}
		number=user_number(i, user_array);
		sum+=number;
		number_display(i, com_array, user_array, sum);
		if (max<sum)
		{
			win=2;
			break;
		}
		press_any_key();
	}
	conclusion(win);
}

void gotoxy(int x, int y)
{
  COORD Pos = {x - 1, y - 1};
  SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}
```

업그레이드 9 : 콘솔 창 설정

[화면/연출]

이전 (문제점)

기본 창 크기와 제목

변경 내용

창 제목을 게임 이름으로, 창 크기를 80x32로 고정

효과

게임 프로그램다운 첫인상

관련 코드 : setup_console()

```
setup_console();
void setup_console(void)
{
    SetConsoleTitleA("100을 넘어라 : 아이템 카드 배틀");
    system("mode con cols=80 lines=32");
}
```

업그레이드 10 : 커서 숨기기

[화면/연출]

이전 (문제점)

화면에서 커서가 계속 깜빡임

변경 내용

CONSOLE_CURSOR_INFO로 커서 숨김

효과

화면이 깔끔해짐

관련 코드 : hide_cursor()

```
hide_cursor();
void hide_cursor(void)
{
    CONSOLE_CURSOR_INFO ci;
    ci.dwSize = 1;
    ci.bVisible = FALSE;

SetConsoleCursorInfo(GetStdHandle(STD_OUTP
UT_HANDLE), &ci);
}
```

업그레이드 11 : 색상 시스템

[화면/연출]

이전 (문제점)

모든 글자가 흰색 한 가지

변경 내용

SetConsoleTextAttribute로 컴퓨터는 빨강, 사용자는 파랑, 경고는 노랑

효과

정보가 한눈에 구분됨

관련 코드 : set_color(), print_at()

```
enum { C_GRAY = 7, C_DARK = 8, C_BLUE = 9,
C_GREEN = 10, C_CYAN = 11,
      C_RED = 12, C_MAGENTA = 13,
C_YELLOW = 14, C_WHITE = 15 };
void set_color(int color)
{

SetConsoleTextAttribute(GetStdHandle(STD_OUT
PUT_HANDLE), (WORD)color);
}
```

업그레이드 12 : 1초 로고(스플래시) 화면

[화면/연출]

이전 (문제점)

실행하자마자 설명 글자만 나옴

변경 내용

큰 100 로고와 1초 로딩 바, 시작 효과음

효과

게임다운 시작 연출

관련 코드 : splash_screen()

```
splash_screen();
void splash_screen(void)
{
    int i;
    system("cls");
    print_at(20, 6,  C_YELLOW, "   ##    ####    ####  ");
    print_at(20, 7,  C_YELLOW, "  ###   ##  ##   ##  ## ");
    print_at(20, 8,  C_YELLOW, "   ##   ##  ##   ##  ## ");
    print_at(20, 9,  C_YELLOW, "   ##   ##  ##   ##  ## ");
    print_at(20, 10, C_YELLOW, "  ####   ####    ####  ");
    print_at(26, 12, C_WHITE,  "을 넘 어 라 !");
    print_at(23, 14, C_CYAN,   "- ITEM CARD BATTLE -");
    print_at(20, 18, C_DARK,   "[                    ]");
    for (i = 0; i < 24; i++) {              /* 약 1초 로딩 바 */
        print_at(21 + i, 18, C_GREEN, "#");
        Sleep(40);
    }
    if (sound_on) { Beep(523, 80); Beep(784, 120); }
}
```

업그레이드 13 : 방향키 메인 메뉴

[화면/연출]

이전 (문제점)

메뉴 없이 바로 게임 시작

변경 내용

새 게임 / 이어하기 / 게임 방법 / 전적 / 소리 / 종료 메뉴

효과

원하는 기능을 골라 실행

관련 코드 : main_menu(), menu_select()

```
sel = main_menu();
int menu_select(int x, int y, const char *items[], int n, int
start)
{
    int sel = start, i, key;
    while (1)
    {
        for (i = 0; i < n; i++) {
            if (i == sel) print_at(x, y + i * 2, C_YELLOW, "▶
%-30s", items[i]);
            else          print_at(x, y + i * 2, C_GRAY,    "
%-30s", items[i]);
        }
        key = get_key();
        if (key == KEY_UP)        { sel = (sel + n - 1) % n;
snd_move(); }
        else if (key == KEY_DOWN) { sel = (sel + 1) % n;
snd_move(); }
        else if (key == KEY_ENTER){ snd_select(); return sel; }
        else if (key == KEY_ESC)  return -1;
    }
}
```

업그레이드 14 : 게임 방법 화면 개편

[화면/연출]

이전 (문제점)

설명 한 화면에 몇 줄만 있음

변경 내용

2페이지 구성 (기본 규칙과 조작법 / 아이템 설명)

효과

처음 하는 사람도 쉽게 이해

관련 코드 : rules_screen()

```
void rules_screen(void)
{
    int i;
    system("cls");
    print_at(3, 2, C_YELLOW, "[ 게임 방법 1/2 : 기본 규칙 ]");
    print_at(3, 4,  C_WHITE, "1. 컴퓨터와 사용자가 번갈아 1~10 숫자 카드를 한
장씩 냅니다.");
    print_at(3, 5,  C_WHITE, "2. 낸 숫자는 합계에 더해지고, 같은 숫자는 다시 낼
수 없습니다.");
    print_at(3, 6,  C_WHITE, "3. 합계를 처음으로 %d보다 크게 만든 사람이
승리합니다!", MAX_SCORE);
    print_at(3, 7,  C_WHITE, "4. 10장을 모두 쓰면 숫자 카드가 다시
채워집니다.");
    print_at(3, 8,  C_WHITE, "5. 선공은 동전 던지기로 정합니다.");
    print_at(3, 9,  C_WHITE, "6. 내 차례에는 %d초 안에 골라야 합니다. (시간
초과 시 무작위)", TIME_LIMIT);
    print_at(3, 12, C_CYAN,  "[ 조작법 ]");
    print_at(3, 13, C_GRAY,  "←→ : 카드 이동     ↑↓ : 숫자 줄 / 아이템 줄
전환");
    print_at(3, 14, C_GRAY,  "Enter : 선택       S : 저장       ESC : 메뉴로");
    press_any_key(20);

    system("cls");
    print_at(3, 2, C_YELLOW, "[ 게임 방법 2/2 : 아이템 카드 ]");
    print_at(3, 4, C_WHITE,  "- 시작할 때 아이템 %d장, %d턴마다 보너스 아이템
1장 (최대 %d장)",
             START_ITEM, BONUS_TURN, ITEM_SLOT);
    print_at(3, 5, C_WHITE,  "- 아이템은 한 턴에 1번, 숫자 카드를 내기 전에
사용합니다.");
    for (i = 1; i < ITEM_KIND; i++)
        print_at(3, 7 + i * 2, C_GREEN, "%s", ITEM_DESC[i]);
    press_any_key(20);
}
```

업그레이드 15 : 카드 모양 UI

[화면/연출]

이전 (문제점)

숫자가 1 5 3처럼 글자로만 나열됨

변경 내용

숫자와 아이템을 상자 모양 카드로 그림. 쓴 카드는 회색, 봉인 카드는 보라색

효과

카드 게임 느낌, 남은 카드가 한눈에 보임

관련 코드 : draw_box(), draw_hand()

```
void draw_box(int x, int y, int w, const char
*text, int color, int selected)
{
    int i;
    set_color(color);
    gotoxy(x, y);
    printf("+");
    for (i = 0; i < w - 2; i++) printf(selected ? "=" :
"-");
    printf("+");
    gotoxy(x, y + 1);
    printf("|%-*s|", w - 2, text);
    gotoxy(x, y + 2);
    printf("+");
    for (i = 0; i < w - 2; i++) printf(selected ? "=" :
"-");
    printf("+");
    set_color(C_GRAY);
}
```

업그레이드 16 : 방향키로 카드 선택

[화면/연출]

이전 (문제점)

scanf로 숫자를 직접 입력

변경 내용

←→로 카드 이동, ↑↓로 숫자/아이템 줄 전환, Enter로 선택

효과

입력 오류가 아예 생길 수 없음

관련 코드 : user_turn(), get_key()

```
#define KEY_UP     (1000 + 72)
#define KEY_DOWN   (1000 + 80)
#define KEY_LEFT   (1000 + 75)
#define KEY_RIGHT  (1000 + 77)
#define KEY_ENTER  13
#define KEY_ESC    27
int get_key(void)
{
    int c = _getch();                    /* [업6]
getch -> _getch */
    if (c == 0 || c == 224)
        return 1000 + _getch();
    return c;
}

void press_any_key(int y)
{
    print_at(3, y, C_DARK, "아무 키나
누르세요...");
    get_key();
}
```

업그레이드 17 : 합계 게이지 바

[화면/연출]

이전 (문제점)

현재 합계가 숫자로만 나옴

변경 내용

[#####] 게이지 표시, 70 이상 노랑, 90 이상 빨강

효과

위기 상황을 바로 알 수 있음

관련 코드 : draw_gauge()

```
#define GAUGE_LEN    40
void draw_gauge(Game *g)
{
    int i, fill = g->sum * GAUGE_LEN / MAX_SCORE;
    int color = g->sum < 70 ? C_GREEN : (g->sum < 90 ? C_YELLOW : C_RED);
    if (fill > GAUGE_LEN) fill = GAUGE_LEN;

    print_at(2, 7, C_WHITE, "합계 %3d / %d  ", g->sum, MAX_SCORE);
    set_color(C_DARK);  printf("[");
    set_color(color);
    for (i = 0; i < GAUGE_LEN; i++) printf(i < fill ? "#" : " ");
    set_color(C_DARK);  printf("]");
    set_color(C_GRAY);
}
```

업그레이드 18 : 화면 깜빡임 줄이기

[화면/연출]

이전 (문제점)

입력할 때마다 화면 전체를 지움(cls)

변경 내용

카드를 움직일 때는 손패 부분만 다시 그림

효과

부드러운 화면

관련 코드 : draw_hand()

```
void draw_hand(Game *g, int row, int col, int item_used)
{
    Player *me = &g->p[USER];
    char buf[20];
    int n, i, color, sel;

    print_at(3, 18, row == 0 && col >= 0 ? C_WHITE : C_DARK, "숫자
카드");
    for (n = MIN_NUM; n <= MAX_NUM; n++) {
        sel = (row == 0 && col == n - 1);
        if (me->used[n])        color = C_DARK;
        else if (me->sealed[n]) color = C_MAGENTA;
        else                    color = C_WHITE;
        if (sel) color = C_YELLOW;
        sprintf(buf, " %2d ", n);
        draw_box(3 + (n - 1) * 7, 19, 6, buf, color, sel);
    }

    print_at(3, 22, row == 1 ? C_WHITE : C_DARK, "아이템 카드%s",
item_used ? " (이번 턴 사용 완료)" : "                    ");
    for (i = 0; i < ITEM_SLOT; i++) {
        sel = (row == 1 && col == i);
        if (me->items[i] == ITEM_NONE || item_used) color = C_DARK;
        else                                        color = C_GREEN;
        if (sel) color = C_YELLOW;
        sprintf(buf, " %s", ITEM_NAME[me->items[i]]);
        draw_box(3 + i * 14, 23, 12, buf, color, sel);
    }
```

업그레이드 19 : 합계 카운트업 애니메이션

[화면/연출]

이전 (문제점)

합계가 한 번에 바뀜

변경 내용

합계가 1씩 올라가며 게이지가 차오름

효과

긴장감 있는 연출

관련 코드 : play_game()

```
for (i = 0; i < value; i++) {
        g->sum++;
        draw_gauge(g);
        Sleep(35);
    }

    if (g->sum > MAX_SCORE)
        return who;

    g->round++;
```

업그레이드 20 : 컴퓨터 생각 중 연출

[화면/연출]

이전 (문제점)

컴퓨터가 순식간에 숫자를 냄

변경 내용

"컴퓨터가 생각 중..." 점 3개 연출

효과

상대와 대결하는 느낌

관련 코드 : computer_turn()

```
int computer_turn(Game *g)
{
    int i;
    show_msg(C_RED, "컴퓨터가 생각 중");
    for (i = 0; i < 3; i++) { printf("."); Sleep(300);
}
    clear_line(29);

    if (g->difficulty == EASY)   return ai_easy(g);
    if (g->difficulty == NORMAL) return
ai_normal(g);
    return ai_hard(g);
}
```

업그레이드 21 : 게임 기록(로그) 창

[화면/연출]

이전 (문제점)

무슨 일이 있었는지 알 수 없음

변경 내용

최근 5개의 행동(카드, 아이템, 보너스)을 표시

효과

흐름을 한눈에 파악

관련 코드 : add_log(), draw_log()

```
#define LOG_LINES    5
char log[LOG_LINES][80];
void add_log(Game *g, const char *fmt, ...)
{
    va_list ap;
    int i;
    for (i = 0; i < LOG_LINES - 1; i++)
        strcpy(g->log[i], g->log[i + 1]);
    va_start(ap, fmt);
    vsnprintf(g->log[LOG_LINES - 1], sizeof(g->log[0]), fmt,
ap);
    va_end(ap);
}

void add_history(Player *p, int card)
{
    int i;
    if (p->history_count == HISTORY_MAX) {
        for (i = 0; i < HISTORY_MAX - 1; i++) p->history[i] =
p->history[i + 1];
        p->history_count--;
    }
    p->history[p->history_count++] = card;
}
```

업그레이드 22 : 효과음 + 소리 ON/OFF

[화면/연출]

이전 (문제점)

소리 없음

변경 내용

이동, 선택, 오류, 아이템, 승리/패배 멜로디 (Beep). 메뉴에서 소리 끄기 가능

효과

생동감 있는 게임

관련 코드 : snd_*() 함수들

```
int sound_on = 1;
void snd_move(void)   { if (sound_on) Beep(600, 25);
}
void snd_select(void) { if (sound_on) Beep(900, 60);
}
void snd_error(void)  { if (sound_on) Beep(200, 150);
}
void snd_card(void)   { if (sound_on) Beep(750, 50);
}
void snd_item(void)   { if (sound_on) { Beep(1200,
70); Beep(1600, 90); } }
void snd_win(void)
{
    int note[ ] = { 523, 659, 784, 1046 }, i;     /* 도 미
솔 도 */
    if (!sound_on) return;
    for (i = 0; i < 4; i++) Beep(note[i], 150);
    Beep(1046, 400);
}
void snd_lose(void)
{
    int note[ ] = { 784, 659, 523, 392 }, i;      /* 솔 미
도 솔 */
    if (!sound_on) return;
    for (i = 0; i < 4; i++) Beep(note[i], 200);
```

업그레이드 23 : 난이도 3단계 AI

[게임 시스템]

이전 (문제점)

컴퓨터가 무조건 무작위로 냄

변경 내용

쉬움: 무작위 / 보통: 이길 수 있으면 이김 / 어려움: 수를 읽고 아이템까지 사용

효과

실력에 맞게 플레이 가능

관련 코드 : select_difficulty(), ai_easy/normal/hard()

```
enum { EASY = 0, NORMAL, HARD };
const char *DIFF_NAME[3] = { "쉬움", "보통",
"어려움" };
const char *WHO_NAME[2]  = { "컴퓨터",
"사용자" };
int select_difficulty(void)
{
    const char *items[3] = { "쉬움   - 아무거나
내는 컴퓨터",
                             "보통   - 이길 수 있으면
이기는 컴퓨터",
                             "어려움 - 수를 읽고
아이템을 쓰는 컴퓨터" };
    system("cls");
    print_at(25, 3, C_YELLOW, "[ 난이도를
선택하세요 ]");
    print_at(25, 20, C_DARK, "ESC : 뒤로");
    return menu_select(18, 7, items, 3, 1);
}
```

업그레이드 24 : 동전 던지기로 선공 정하기

[게임 시스템]

이전 (문제점)

항상 컴퓨터가 먼저 시작 (불공평)

변경 내용

앞면/뒷면을 맞히면 사용자 선공, 동전 회전 애니메이션

효과

공정성과 재미 추가

관련 코드 : coin_toss()

```
int coin_toss(void)
{
    const char *items[2] = { "앞면", "뒷면" };
    int guess, result, i;

    system("cls");
    print_at(22, 3, C_YELLOW, "[ 동전 던지기로 선공을 정합니다 ]");
    print_at(22, 5, C_GRAY,   "앞면과 뒷면 중 하나를 고르세요.");
    guess = menu_select(32, 8, items, 2, 0);
    if (guess < 0) guess = 0;

    result = rand() % 2;
    for (i = 0; i < 12; i++) {                    /* 동전 돌아가는 연출 */
        print_at(33, 14, (i % 2) ? C_YELLOW : C_WHITE, "(  %s  )", items[i % 2]);
        if (sound_on) Beep(1000, 20);
        Sleep(60 + i * 15);
    }
    print_at(33, 14, C_YELLOW, "(  %s  )", items[result]);
    if (guess == result) print_at(27, 17, C_BLUE, "맞혔습니다! 사용자가 먼저 합니다.");
    else                 print_at(27, 17, C_RED,  "틀렸습니다! 컴퓨터가 먼저 합니다.");
    Sleep(1500);
    return (guess == result) ? USER : COM;
}

void new_game(Game *g, int difficulty)
{
    int i, w;
    memset(g, 0, sizeof(Game));
    g->magic = SAVE_MAGIC;
    g->difficulty = difficulty;
```

업그레이드 25 : 제한 시간 20초

[게임 시스템]

이전 (문제점)

시간 제한 없음

변경 내용

_kbhit()와 clock()으로 타이머, 5초 이하는 빨간색과 경고음, 시간이 지나면 무작위 선택

효과

긴장감 추가

관련 코드 : user_turn()

```
#define TIME_LIMIT   20

int user_turn(Game *g)
{
    Player *me = &g->p[USER];
    int row = 0, col = max_playable(me) - 1;
    int item_used = 0, key, remain, last_sec = -1;
    clock_t start = clock(), pause_start;

    draw_hand(g, row, col, item_used);
    while (1)
    {
 remain = TIME_LIMIT - (int)((clock() - start) / CLOCKS_PER_SEC);
        if (remain != last_sec) {
            last_sec = remain;
            print_at(3, 28, remain <= 5 ? C_RED : C_GRAY, "남은 시간 :
%2d초 ", remain < 0 ? 0 : remain);
            if (remain <= 5 && remain > 0 && sound_on) Beep(1000, 30);
        }
        if (remain <= 0) {
            int n = random_playable(me);
            add_log(g, "시간 초과! 사용자 카드가 무작위로 선택됐습니다.");
            snd_error();
            return n;
        }
```

업그레이드 26 : 숫자 카드 리필 규칙

[게임 시스템]

이전 (문제점)

카드가 떨어지면 게임 진행 불가 (아이템 추가 후 문제)

변경 내용

10장을 모두 쓰면 숫자 카드를 다시 채움

효과

무승부 없이 항상 승부가 남

관련 코드 : play_game()

```
if (all_used(me)) {
        memset(me->used, 0,
sizeof(me->used));
        add_log(g, "%s의 숫자 카드를 모두 써서
다시 채웠습니다!", WHO_NAME[who]);
      }

      draw_game(g);
```

업그레이드 27 : 아이템 카드 시스템 도입

[아이템 배틀]

이전 (문제점)

숫자만 내는 단순한 게임

변경 내용

아이템 슬롯 3칸, 시작 2장, 숫자를 내기 전 한 턴에 1번 사용

효과

전략성이 크게 늘어남

관련 코드 : Player 구조체, use_item()

```
#define ITEM_SLOT    3
#define START_ITEM   2
enum { ITEM_NONE = 0, ITEM_DOUBLE, ITEM_MINUS, ITEM_SKIP,
ITEM_SEAL, ITEM_KIND };
const char *ITEM_NAME[ITEM_KIND] = { "(빈칸)", "더블 x2", "합계
-10", "턴 스킵", "봉인" };
const char *ITEM_DESC[ITEM_KIND] = {
    "",
    "[더블 x2] 이번 턴에 내는 숫자가 2배가 됩니다.",
    "[합계 -10] 현재 합계를 10 줄입니다. (상대의 승리 저지용)",
    "[턴 스킵] 상대의 다음 턴을 건너뛰고 내가 한 번 더 합니다.",
    "[봉인] 상대의 가장 큰 숫자 카드를 다음 1턴 동안 못 쓰게 합니다."
};
for (w = 0; w < 2; w++)
for (i = 0; i < START_ITEM; i++)
            g->p[w].items[i] = rand() % (ITEM_KIND - 1) + 1;
    g->turn = coin_toss();
    add_log(g, "게임 시작! 난이도: %s, 선공: %s",
DIFF_NAME[difficulty], WHO_NAME[g->turn]);
}

void run_game(Game *g)
{
    int winner = play_game(g);
    if (winner >= 0)
        ending_screen(g, winner);
}
```

업그레이드 28 : 아이템 1: 더블 x2

[아이템 배틀]

이전 (문제점)

–

변경 내용

이번 턴에 내는 숫자가 2배

효과

역전의 한 방

관련 코드 : use_item() ITEM_DOUBLE

```
int double_on;
 value = card * (g->double_on ? 2 : 1);
 me->used[card] = 1;
void use_item(Game *g, int who, int slot)
{
    Player *me = &g->p[who], *opp = &g->p[1
- who];
    int type = me->items[slot], n;

    me->items[slot] = ITEM_NONE;
    me->item_used_total++;
    snd_item();

    switch (type) {
```

업그레이드 29 : 아이템 2: 합계 -10

[아이템 배틀]

이전 (문제점)

-

변경 내용

현재 합계를 10 줄임 (0 미만 방지)

효과

상대의 승리를 막는 방어 카드

관련 코드 : use_item() ITEM_MINUS

```
#define MINUS_VALUE  10

void use_item(Game *g, int who, int slot)
{
    Player *me = &g->p[who], *opp = &g->p[1
- who];
    int type = me->items[slot], n;

    me->items[slot] = ITEM_NONE;
    me->item_used_total++;
    snd_item();

    switch (type) {
```

업그레이드 30 : 아이템 3: 턴 스킵

[아이템 배틀]

이전 (문제점)

–

변경 내용

상대 턴을 건너뛰고 내가 한 번 더 함

효과

연속 공격 가능

관련 코드 : use_item() ITEM_SKIP

```
int skip_next;

 if (g->skip_next) {
        g->skip_next = 0;
           add_log(g, "턴 스킵 효과! %s가 한 번 더 합니다.",
WHO_NAME[who]);
        } else {
           g->turn = 1 - who;
        }
        draw_log(g);
        Sleep(600);
    }
}
void use_item(Game *g, int who, int slot)
{
    Player *me = &g->p[who], *opp = &g->p[1 - who];
    int type = me->items[slot], n;

    me->items[slot] = ITEM_NONE;
    me->item_used_total++;
    snd_item();

    switch (type) {
```

업그레이드 31 : 아이템 4: 봉인

[아이템 배틀]

이전 (문제점)

–

변경 내용

상대의 가장 큰 숫자 카드를 1턴 동안 못 쓰게 함 (보라색 표시)

효과

상대의 결정타 차단

관련 코드 : use_item() ITEM_SEAL

```
int sealed[MAX_NUM + 1];
 if (max_playable(me) == 0) {
            add_log(g, "%s: 낼 수 있는 카드가 없어 턴을 넘깁니다.",
WHO_NAME[who]);
            memset(me->sealed, 0, sizeof(me->sealed));
            g->turn = 1 - who;
            draw_log(g);
            Sleep(1000);
            continue;
        }

        if (who == USER) {
            card = user_turn(g);

 memset(me->sealed, 0, sizeof(me→sealed));
void use_item(Game *g, int who, int slot)
{
    Player *me = &g->p[who], *opp = &g->p[1 - who];
    int type = me->items[slot], n;

    me->items[slot] = ITEM_NONE;
    me->item_used_total++;
    snd_item();

    switch (type) {
```

업그레이드 32 : 보너스 아이템 지급

[아이템 배틀]

이전 (문제점)

아이템이 떨어지면 끝

변경 내용

4턴마다 양쪽 빈 슬롯에 아이템 1장 지급

효과

게임 후반까지 전략 유지

관련 코드 : give_bonus()

```
#define BONUS_TURN   4
 if (g->round % BONUS_TURN == 0)
       give_bonus(g);
void give_bonus(Game *g)
{
    int w, i;
    for (w = 0; w < 2; w++)
        for (i = 0; i < ITEM_SLOT; i++)
            if (g->p[w].items[i] == ITEM_NONE) {
                g->p[w].items[i] = rand() %
(ITEM_KIND - 1) + 1;
                break;
            }
    add_log(g, "%d턴 보너스! 양쪽에 아이템 1장씩
지급 (빈 슬롯이 있을 때)", BONUS_TURN);
    if (sound_on) { Beep(880, 60); Beep(1320,
60); }
}
```

업그레이드 33 : AI 아이템 전략

[아이템 배틀]

이전 (문제점)

컴퓨터는 아이템을 쓸 줄 모름

변경 내용

어려움 AI: 이길 수 있으면 더블, 위기면 봉인, -10, 스킵 순서로 방어

효과

똑똑한 상대와 대결

관련 코드 : ai_hard(), safe_card()

```
int playable(Player *p, int n) { return !p->used[n] && !p->sealed[n]; }

int max_playable(Player *p)
{
    int n;
    for (n = MAX_NUM; n >= MIN_NUM; n--) if (playable(p, n)) return n;
    return 0;
}

int min_playable(Player *p)
{
    int n;
    for (n = MIN_NUM; n <= MAX_NUM; n++) if (playable(p, n)) return n;
    return 0;
}

int random_playable(Player *p)
{
    int list[MAX_NUM], cnt = 0, n;
    for (n = MIN_NUM; n <= MAX_NUM; n++) if (playable(p, n)) list[cnt++] = n;
    return cnt ? list[rand() % cnt] : 0;
}

int all_used(Player *p)
{
    int n;
    for (n = MIN_NUM; n <= MAX_NUM; n++) if (!p->used[n]) return 0;
    return 1;
}

int find_item(Player *p, int type)
{
    int i;
    for (i = 0; i < ITEM_SLOT; i++) if (p->items[i] == type) return i;
    return -1;
}

int item_count(Player *p)
{
    int i, cnt = 0;
    for (i = 0; i < ITEM_SLOT; i++) if (p->items[i] != ITEM_NONE) cnt++;
    return cnt;
}
```

업그레이드 34 : 게임 저장

[저장/마무리]

이전 (문제점)

게임을 끄면 진행이 모두 사라짐

변경 내용

내 차례에 S 키를 누르면 save.dat에 저장 (fwrite)

효과

언제든 중단 가능

관련 코드 : save_game()

```
#define SAVE_FILE    "save.dat"
 else if (key == 's' || key == 'S') {
if (item_used)        show_msg(C_RED, "아이템을
쓴 턴에는 저장할 수 없습니다.");
            else if (save_game(g))
show_msg(C_GREEN, "게임을 저장했습니다!
(메뉴 > 이어하기)");
            else                  show_msg(C_RED,
"저장에 실패했습니다.");
        }
```

업그레이드 35 : 이어하기(불러오기)

[저장/마무리]

이전 (문제점)

–

변경 내용

메뉴의 이어하기로 저장한 게임 불러오기, 잘못된 파일인지 확인 (magic 번호)

효과

중단한 곳부터 다시 시작

관련 코드 : load_game()

```
#define SAVE_MAGIC   20261105
int magic;
int save_game(Game *g)
{
    FILE *fp = fopen(SAVE_FILE, "wb");
    if (fp == NULL) return 0;
    fwrite(g, sizeof(Game), 1, fp);
    fclose(fp);
    return 1;
}

int load_game(Game *g)
{
    FILE *fp = fopen(SAVE_FILE, "rb");
    int ok;
    if (fp == NULL) return 0;
    ok = (fread(g, sizeof(Game), 1, fp) == 1) && (g->magic == SAVE_MAGIC);
    fclose(fp);
    if (ok) add_log(g, "저장된 게임을 불러왔습니다.");
    return ok;
}

void load_record(Record *r)
{
    FILE *fp = fopen(RECORD_FILE, "r");
    r->win = r->lose = r->draw = 0;
    if (fp == NULL) return;
    if (fscanf(fp, "%d %d %d", &r->win, &r->lose, &r->draw) != 3)
        r->win = r->lose = r->draw = 0;
    fclose(fp);
}

void save_record(Record *r)
{
    FILE *fp = fopen(RECORD_FILE, "w");
    if (fp == NULL) return;
    fprintf(fp, "%d %d %d₩n", r->win, r->lose, r->draw);
    fclose(fp);
}
```

업그레이드 36 : 전적 기록 저장/보기

[저장/마무리]

이전 (문제점)

승패 기록 없음

변경 내용

record.txt에 승/패/무를 저장하고 메뉴에서 승률까지 표시

효과

계속 도전하게 만드는 동기

관련 코드 : load_record(), save_record(), record_screen()

```
#define RECORD_FILE  "record.txt"
void record_screen(void)
{
    Record r;
    int total;
    load_record(&r);
    total = r.win + r.lose + r.draw;
    system("cls");
    print_at(30, 5, C_YELLOW, "[ 나의 전적 ]");
    print_at(30, 8,  C_BLUE,  "승리 : %3d",
r.win);
    print_at(30, 10, C_RED,   "패배 : %3d",
r.lose);
    print_at(30, 12, C_GRAY,  "무승부 : %3d",
r.draw);
    print_at(30, 14, C_GREEN, "승률 :
%5.1f%%", total ? r.win * 100.0 / total : 0.0);
    press_any_key(20);
}
```

업그레이드 37 : 엔딩 화면

[저장/마무리]

이전 (문제점)

"사용자가 이겼습니다" 한 줄

변경 내용

YOU WIN / YOU LOSE 큰 글자, 결과 통계(턴, 합계, 아이템), 승리/패배 멜로디

효과

완성도 있는 마무리

관련 코드 : ending_screen()

```
 int item_used_total;
void ending_screen(Game *g, int winner)
{
    Record r;
    int color = (winner == USER) ? C_YELLOW : C_RED;

    load_record(&r);                          /* [업36] 전적 저장 */
    if (winner == USER) r.win++;
    else if (winner == COM) r.lose++;
    else r.draw++;
    save_record(&r);
    remove(SAVE_FILE);                        /* 끝난 게임은 이어하기 불가 */

    system("cls");
    if (winner == USER) {
        print_at(18, 3, color, " #   #  ### #   #    #   # ### #   #");
        print_at(18, 4, color, "  # #  #  # #  #    #   #  #  ## #");
        print_at(18, 5, color, "   #   #  # #  #    # # #  #  # # #");
        print_at(18, 6, color, "   #   #  # #  #    ## ##  #  # ##");
        print_at(18, 7, color, "   #   ###  ###     #  # ### #   #");
        print_at(30, 9, C_WHITE, "사용자가 이겼습니다!");
    } else if (winner == COM) {
        print_at(15, 3, color, " #   #  ### #   #    #    ###  #### #####");
        print_at(15, 4, color, "  # #  #  # #  #    #    #  # #    #   ");
        print_at(15, 5, color, "   #   #  # #  #    #    #  # ###  #### ");
        print_at(15, 6, color, "   #   #  # #  #    #    #  #   # #   ");
        print_at(15, 7, color, "   #   ###  ###     ####  ### ####  #####");
        print_at(30, 9, C_WHITE, "컴퓨터가 이겼습니다...");
    } else {
        print_at(33, 6, C_GRAY, "무승부입니다.");
    }
```

업그레이드 38 : 일시정지 + 다시 하기

[저장/마무리]

이전 (문제점)

한 판 하면 프로그램 종료

변경 내용

ESC로 일시정지와 메뉴 복귀(타이머도 멈춤), 게임이 끝나면 메뉴로 돌아와 다시 하기

효과

여러 판 연속 플레이

관련 코드 : main() while 반복, user_turn() ESC

```
While (1)
{
if (card == 0) return -1;
 } else {
          card = computer_turn(g);
       }

 else if (key == KEY_ESC) {
 pause_start = clock();
          show_msg(C_YELLOW, "일시정지 –
메뉴로 나갈까요? 저장 안 한 진행은 사라집니다
(Y/N)");
          key = get_key();
          if (key == 'y' || key == 'Y') return 0;
```

정리

버그 수정 8회 → 화면/연출 14회 → 게임 시스템 4회 → 아이템 배틀 7회 → 저장/마무리 5회
단순한 숫자 더하기 게임을 전략적인 아이템 카드 배틀로 확장했습니다.

점수 : 30/29